

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

Kumbalagodu, Mysore Road, Bengaluru-560074

A Project Report on

AthleteForge

(2025-2026)

BANGALORE UNIVERSITY (JNANABHARATHI)

UNDER THE GUIDANCE OF

Ms. Dakshayini L & Mr. Madhusudan N

Dept. of BCA, DBIMSCA.

SUBMITTED BY

ROHITH GOWDA V & PRAKRUTHI R

(U03CQ23S0045 & U03LK23S0100)

BANGALORE UNIVERSITY (JNANABHARATHI)

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

This is to certify that ROHITH GOWDA V & PRAKRUTHI R bearing reg.no U03CQ23S0045 & U03LK23S0100 has successfully completed the project entitled "AthleteForge" under the guidance of Ms. Dakshayini L & Mr. Madhusudan N, for the partial fulfilment of BCA Sixth semester course in the computer laboratory at DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS, during the period of March 2026 to June 2026.

Project Guide HOD Director Principal

Prof. Dakshayini L Prof. Deepa T Dr. Veena R Prof. H Pandurangappa

Prof. Madhusudan N

Signature of the Examiners:

1) _____

2) _____

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

Kumbalagodu, Mysore Road, Bengaluru-560074

Student Declaration

We, ROHITH GOWDA V & PRAKRUTHI R, hereby declare that this project report entitled "AthleteForge" for the partial fulfilment of BCA Sixth semester course in the Computer Laboratory at DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS, during the period of March 2026 to June 2026.

Signature

1. ROHITH GOWDA V

2. PRAKRUTHI R

ACKNOWLEDGEMENT

We hereby acknowledge, with regards and respects, the encouragement and guidance extended by Asst.Prof. Dakshayini L & Asst.Prof. Madhusudan N, faculty, Dept. of Computer Applications, DBIMSCA, Bangalore University, Jnanabharathi, Bengaluru.

We use this opportunity to thank Professor Mrs. Deepa T, HOD, Dept. of Computer Applications, DBIMSCA, Bangalore University, especially for providing the platform to develop skills of presentation of Synopsis and Project.

We use this opportunity to thank Dr. Veena R, Director of Computer Applications, DBIMSCA, Bangalore University.

Finally, we would like to thank our Principal Prof. H Pandurangappa, DBIMSCA, Bangalore University and teachers of the department for their support.

Project Associates:

1. ROHITH GOWDA V

2. PRAKRUTHI R

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Project Overview

1.2 Aims and Objectives

1.3 Scope of the Project

1.4 Applications

1.5 Background of the Project

1.6 Problem Statement

1.7 Features of the Project

1.8 Advantages of the System

1.9 Limitations of the System

1.10 Summary of Chapter

CHAPTER 2 SYSTEM ANALYSIS 8

2.1 Introduction to System Analysis

2.2 Requirement Analysis

2.3 Existing System

2.4 Proposed System

2.5 Software Tools Used

2.6 Feasibility Study

2.7 Constraints of the System

2.8 Risk Analysis

2.9 Development Methodology

2.10 Summary of Chapter

CHAPTER 3 SYSTEM DESIGN 18

3.1 Introduction to System Design

3.2 Overall System Architecture

3.3 Table Design

3.4 Data Flow Diagrams

3.5 Use Case Diagram

3.6 Software Design

3.7 Flowchart Presentation

3.8 Summary of Chapter

CHAPTER 4 SYSTEM TESTING 32

4.1 Introduction to Testing

4.2 Types of Testing

4.3 Test Cases

4.4 Error Handling and Debugging

4.5 Summary of Chapter

CHAPTER 5 SYSTEM IMPLEMENTATION 38

CHAPTER 6 FUTURE SCOPE 44

CHAPTER 7 CONCLUSION 48

CHAPTER 8 REFERENCES 50

APPENDIX A VIVA QUESTIONS AND ANSWERS 52

APPENDIX B INSTALLATION GUIDE 54

APPENDIX C GLOSSARY OF TERMS 56

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

In recent years, sports science and digital technology have transformed how coaches, academies, and educational institutions manage athlete performance. The increasing demand for centralized data systems that track training metrics, injuries, attendance, and competition results has led to the development of intelligent web-based sports management platforms.

AthleteForge is a full-stack web application designed for sports academies, college teams, and coaching centres. The system enables administrators, coaches, and student athletes to manage performance data through a secure role-based portal with real-time analytics dashboards and AI-assisted coaching insights.

The frontend is built using React 18 with Vite, providing a fast and responsive Single Page Application (SPA). The backend uses Django 4.2 with Django REST Framework (DRF) to expose secure REST APIs. MySQL 8.0 stores athlete records during local development, while PostgreSQL is used on the Render.com cloud deployment.

Key modules include Athlete Management, Performance Tracking across five fitness dimensions (speed, strength, endurance, flexibility, agility), Injury Tracking with recovery workflow, Competition and Medal Management, Attendance with bulk marking, Weight Monitoring with automatic BMI calculation, PDF/Excel Reports, and AI-powered Readiness Copilot with optional voice interaction.

The live deployment is hosted at <https://athleteforge-bca.onrender.com> on Render.com cloud platform. The project repository is maintained on GitHub at <https://github.com/rohith917/athleteforge-bca>.

ATHLETEFORGE DBIMSCA

1.2 AIMS AND OBJECTIVES

Aim of the Project

The primary aim of this project is to develop a comprehensive web-based Athlete Performance and Injury Tracking System that enables sports institutions to manage athlete data efficiently and make informed coaching decisions using analytics and AI-assisted insights.

Objectives of the Project

To design and develop a secure multi-role authentication system supporting Admin, Coach, and Student roles

To implement complete CRUD operations for athletes, performance records, injuries, competitions, attendance, and weight tracking

To build interactive dashboards with Chart.js visualizations showing KPIs, trends, and risk indicators

To integrate AI readiness scoring, injury risk alerts, and conversational copilot for coaching guidance

To generate professional PDF and Excel reports for administrative and academic records

To deploy the application on cloud infrastructure with same-origin API and frontend serving

To demonstrate practical application of BCA curriculum concepts including DBMS, web technologies, and software engineering

ATHLETEFORGE DBIMSCA

1.3 SCOPE OF THE PROJECT

Current Scope

Web-based athlete management for sports academies and college departments

Role-based dashboards for Admin, Coach, and Student with route-level access control

Performance tracking across five standardized fitness metrics with historical trends

Complete injury lifecycle management from reporting through recovery status updates

Competition scheduling, result entry, and medal tracking

Bulk attendance marking with monthly analytics and session type classification

Weight monitoring with automatic BMI calculation on data entry

AI insights panel, Tech Command Hub, and floating AI Copilot widget

Server-side PDF (ReportLab) and Excel (OpenPyXL) report generation

Future Scope

Wearable device integration for live heart rate and GPS training data

Native mobile application for on-field data entry

Machine learning based injury prediction models

Video analysis for biomechanics assessment

Multi-academy tenancy with isolated data partitions

Limitations of Scope

Requires stable internet connection for cloud deployment access

English language interface only in current version

Single academy deployment per service instance

AI LLM responses require optional Groq or Gemini API keys

ATHLETEFORGE DBIMSCA

1.4 APPLICATIONS OF ATHLETEFORGE

Sports Academies: Centralized tracking of all enrolled athletes across multiple sports disciplines including cricket, athletics, football, and basketball. Coaches can monitor team readiness before tournaments.

College Sports Departments: Management of inter-collegiate teams, attendance compliance for sports scholarship students, and official competition records for university audits.

Professional Coaching: Readiness monitoring before matches, injury risk identification through AI analytics, and data-driven training plan adjustments.

Athlete Self-Service: Students view personal performance trends, injury recovery status, and AI-generated training guidance through a dedicated student hub dashboard.

Sports Medicine and Physiotherapy: Digital injury records with severity classification, body part mapping, and recovery timeline tracking for medical staff reference.

Academic Demonstration: Serves as a comprehensive BCA final-year project demonstrating full-stack development, database design, testing, and cloud deployment.

ATHLETEFORGE DBIMSCA

1.5 BACKGROUND OF THE PROJECT

Traditional sports management in most Indian colleges and academies relies on paper registers, handwritten attendance sheets, and disconnected Excel spreadsheets. These methods are difficult to search, prone to calculation errors, lack real-time analytics, and provide no student self-service portal.

Commercial sports management platforms such as TeamSnap and Hudl exist internationally but require expensive subscriptions unsuitable for academic institutions and small coaching centres in India.

Modern open-source web technologies make it feasible to build professional-grade systems as BCA final-year projects. React provides component-based interactive UIs, Django offers robust server-side logic with built-in security, and Chart.js enables rich data visualizations without proprietary licenses.

The integration of AI features - readiness scoring, injury risk alerts, and conversational copilot - adds contemporary relevance and strengthens the viva demonstration value of this academic project.

This project demonstrates how a three-tier web architecture can solve practical sports data management challenges while meeting Bangalore University BCA sixth semester project

requirements.

ATHLETEFORGE DBIMSCA

1.6 PROBLEM STATEMENT

Problems Identified in Existing Manual Systems:

Fragmented athlete data spread across multiple Excel files and paper registers

No real-time visibility into individual or team performance trends

Manual report generation for attendance and performance is time-consuming

Students have no access to their own performance history or injury records

No automated BMI calculation or attendance percentage analytics

Coaches cannot quickly identify injury-prone athletes before competitions

No centralized audit trail for competition results and medal achievements

Proposed Solution:

AthleteForge provides a unified web platform with role-based access control, automated dashboard analytics, server-side report generation, AI-assisted coaching insights, and cloud deployment for 24/7 accessibility from any device with a web browser.

ATHLETEFORGE DBIMSCA

1.7 FEATURES OF THE PROJECT

Multi-role authentication with Admin, Coach, and Student dashboards

Athlete CRUD with photo upload, sport assignment, and team grouping

Five-metric performance tracking with radar charts and trend lines

Injury management with severity levels, body part, and recovery status workflow

Competition records with medal and position tracking per athlete

Bulk attendance marking for entire team in single operation

Weight tracking with automatic $BMI = \text{weight(kg)} / \text{height(m)}^2$ calculation

Coach dashboard with injury heatmap, readiness gauge, and wellness check-in

Student hub with personal performance radar and training tips

Admin panel for user management, system statistics, and module access

AI Readiness Copilot with voice input and text-to-speech coaching tips

PDF and Excel report export for athletes, performance, injuries, and attendance

Responsive Bootstrap 5 layout for desktop and mobile browsers

Same-origin deployment on Render.com with health check endpoint

ATHLETEFORGE DBIMSCA

1.8 ADVANTAGES OF THE SYSTEM

Centralized database eliminates data duplication and inconsistency

Real-time Chart.js visualizations improve coaching decision quality

Role-based security protects sensitive medical and performance data

Open-source technology stack requires no software licensing cost

Cloud deployment on Render.com provides free-tier public accessibility

AI features demonstrate integration of modern AI in web applications

Automated BMI and attendance calculations reduce manual errors

One-click PDF/Excel reports save hours of manual document preparation

Demo accounts enable immediate evaluation during viva presentation

1.9 LIMITATIONS OF THE SYSTEM

Requires stable broadband internet for cloud-hosted version

AI copilot LLM responses need optional GROQ_API_KEY or GEMINI_API_KEY environment variables

No native Android or iOS mobile application in current version

English language interface only; no Kannada or Hindi localization

Render free tier causes 30-60 second cold start after inactivity

Single academy per deployment; no multi-tenant organization support

1.10 SUMMARY OF CHAPTER

This chapter introduced AthleteForge, a comprehensive Athlete Performance and Injury Tracking System. It covered the project aims, scope, real-world applications, background context, problem statement, detailed features, advantages, and limitations. The system addresses genuine sports management challenges using modern full-stack web development with AI integration and cloud deployment.

ATHLETEFORGE DBIMSCA

CHAPTER 2

SYSTEM ANALYSIS

2.1 INTRODUCTION TO SYSTEM ANALYSIS

System Analysis is a crucial phase in software project development that involves understanding user requirements, identifying problems in existing methods, and proposing effective software solutions. It defines how the system should function and what resources are required for successful implementation.

In the context of AthleteForge, system analysis focused on the needs of three user groups: administrators who manage users and system configuration, coaches who manage athlete rosters and training data, and students who need personal performance visibility and injury status updates.

This chapter provides detailed analysis of system requirements, comparison of existing manual systems with the proposed web platform, software tools selection, feasibility study, system constraints, and risk mitigation strategies.

ATHLETEFORGE DBIMSCA

2.2 REQUIREMENT ANALYSIS

2.2.1 Hardware Requirements

Component	Minimum Specification	Recommended
Processor	Intel Core i3 or AMD equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	2 GB free disk space	5 GB SSD
Display	1366 x 768 resolution	1920 x 1080 Full HD
Network	Broadband 2 Mbps	Broadband 10 Mbps or higher
Input Device	Keyboard and Mouse	Keyboard, Mouse, Microphone (for voice AI)

2.2.2 Software Requirements

Software	Version	Purpose
Python	3.10 or higher	Django backend runtime
Node.js	18 LTS or higher	React frontend build toolchain
MySQL	8.0	Local development database
PostgreSQL	15+	Render cloud production database
Django	4.2	Web framework and ORM

Django REST Framework	3.14+	REST API layer
React	18	Single Page Application UI
Vite	5	Frontend build and dev server
Chart.js	4.x	Dashboard data visualizations
Visual Studio Code	Latest	Integrated development environment
Google Chrome / Edge	Latest	Browser testing and PDF viewing

2.2.3 Functional Requirements

ID	Requirement	Priority
FR01	User login with role-based session authentication	High
FR02	Admin can create, update, and deactivate user accounts	High
FR03	Coach can add and edit athlete profiles with photos	High
FR04	Coach can record five-metric performance scores per session	High
FR05	Coach can log injuries with severity and recovery tracking	High
FR06	Coach can mark bulk attendance for team sessions	High
FR07	Student can view personal dashboard linked to athlete profile	High
FR08	System generates PDF and Excel reports on demand	Medium
FR09	AI copilot answers training and readiness questions	Medium
FR10	Dashboard displays Chart.js analytics for each role	High
FR11	Weight entry auto-calculates BMI value	Medium
FR12	Competition results track medals and positions	Medium

ATHLETEFORGE DBIMSCA

2.3 EXISTING SYSTEM

The existing approach in most sports academies and college departments uses manual registers and Excel spreadsheets managed by individual coaches.

Characteristics of Existing System:

Paper-based or Excel athlete records stored on individual computers
No dedicated student self-service portal for performance viewing
Manual chart creation in Excel for performance trend analysis
No integrated injury recovery workflow with status tracking
Attendance maintained in separate registers per sport discipline
Reports created manually by copying data into Word or Excel templates
No AI-assisted readiness or injury risk analysis capability

Limitations of Existing System:

Data duplication across multiple files leads to inconsistency
No real-time analytics or dashboard visualization
Risk of permanent data loss from hardware failure or file corruption
Time-consuming manual report preparation before college audits
Students cannot independently verify their attendance or performance records
No centralized competition and medal achievement database

ATHLETEFORGE DBIMSCA

2.4 PROPOSED SYSTEM

AthleteForge is a web-based Athlete Performance and Injury Tracking System with multi-role dashboards, REST API backend, and AI-assisted analytics replacing manual processes.

Features of Proposed System:

Centralized relational database (MySQL local / PostgreSQL cloud)
Role-based authentication with session cookies and CSRF protection
Automated Chart.js visualizations on coach and student dashboards
Digital injury and recovery tracking with medical notes field
One-click PDF and Excel report generation from server
AI readiness copilot with rule-based analytics and optional LLM enhancement
Same-origin cloud deployment serving React SPA and API from single URL
Health check endpoint for production monitoring

Feature	Existing System	Proposed System
Data Storage	Excel / Paper registers	MySQL / PostgreSQL Database
User Access	Shared files on one PC	Role-based web login
Analytics	Manual Excel charts	Automated Chart.js dashboards
Reports	Manual copy-paste	One-click PDF / Excel export
AI Support	Not available	Readiness Copilot + Insights
Student Portal	Not available	Personal Student Hub dashboard
Attendance	Paper register	Digital bulk marking + reports
Deployment	Local files only	Cloud URL accessible 24/7

ATHLETEFORGE DBIMSCA

2.5 SOFTWARE TOOLS USED

Tool	Category	Role in Project
React + Vite	Frontend Framework	SPA user interface and routing
Bootstrap 5 + React-Bootstrap	CSS Framework	Responsive grid and components
Django + DRF	Backend Framework	REST API, ORM, session auth
MySQL / PostgreSQL	Database	Relational data persistence
Chart.js + react-chartjs-2	Visualization	Dashboard charts and gauges
Axios	HTTP Client	API communication with CSRF headers
ReportLab	PDF Library	Server-side PDF report generation
OpenPyXL	Excel Library	Server-side XLSX export
Gunicorn	WSGI Server	Production application server
WhiteNoise	Static Files	Serves React build in production
Render.com	Cloud Platform	Production hosting and PostgreSQL
Git + GitHub	Version Control	Source code management and CI

ATHLETEFORGE DBIMSCA

2.6 FEASIBILITY STUDY

2.6.1 Technical Feasibility

All selected technologies are mature, open-source, and extensively documented with active community support. Django 4.2 provides built-in security features including CSRF protection, SQL injection prevention through ORM, and password hashing. React 18 with Vite enables rapid frontend development with hot module replacement during coding.

The development team successfully built all modules, tested locally using START_DEMO.bat on Windows, and deployed to Render.com cloud platform. The health endpoint at /api/health/ confirms production availability. Technical feasibility is fully confirmed.

2.6.2 Economic Feasibility

Total project cost is minimal. All frameworks (Django, React, Chart.js) are free and open-source. Render.com provides free-tier web service and PostgreSQL database suitable for academic demonstration. No proprietary software licenses are required.

Development hardware requirements are standard laptop specifications available to all BCA students. The project is economically feasible within academic budget constraints.

2.6.3 Operational Feasibility

The user interface follows familiar web application patterns with navigation sidebar, form inputs, data tables, and chart widgets. Demo accounts (admin/admin123, coach/coach123, rahul.sharma@email.com/student123) enable immediate system evaluation without setup.

START_DEMO.bat automates Windows environment setup by starting Django backend and Vite frontend simultaneously. Operational feasibility is high for coaches, students, and administrators with basic computer literacy.

ATHLETEFORGE DBIMSCA

2.7 CONSTRAINTS OF THE SYSTEM

Internet dependency for cloud-hosted version on Render.com

English-only user interface in current implementation

AI LLM features require optional third-party API keys (Groq or Gemini)

Render free tier imposes cold start delay of 30-60 seconds

File upload size limited by server configuration for athlete photos

Single organization per deployment instance

Browser compatibility requires modern JavaScript-enabled browser

ATHLETEFORGE DBIMSCA

2.8 RISK ANALYSIS

Possible Risks:

Risk	Impact	Probability
Database connection failure	High	Low
Session expiry during data entry	Medium	Medium

Cloud service cold start timeout	Medium	High on free tier
CSRF token mismatch on API calls	Medium	Low
Incorrect BMI from invalid height input	Low	Low
Unauthorized API access attempt	High	Low

Risk Mitigation Strategies:

Database connection pooling and environment variable validation in settings.py

Axios interceptor refreshes CSRF token before mutating API requests

Frontend wakeServer() function pings health endpoint before login on cloud

Server-side input validation in DRF serializers for all data fields

IsAuthenticated permission class on all protected API endpoints

Graceful error messages displayed via Toast notification component

ATHLETEFORGE DBIMSCA

2.9 DEVELOPMENT METHODOLOGY

The project followed the Software Development Life Cycle (SDLC) with iterative refinement at each phase:

Phase	Duration	Activities Completed
Planning	Week 1-2	Topic selection, synopsis preparation, guide approval
Analysis	Week 3-4	Requirement gathering, existing system study, feasibility
Design	Week 5-7	Architecture design, ER diagram, API planning, UI wireframes
Implementation	Week 8-14	Backend models, API endpoints, React pages, AI module
Testing	Week 15-16	25 test cases, bug fixes, error handling improvements
Deployment	Week 17	Render.com setup, PostgreSQL, production verification
Documentation	Week 18	Project report, viva preparation, demo rehearsal

An agile-inspired iterative approach was used within each phase. Backend API endpoints were tested with browser and Postman before frontend integration. Each dashboard module was demonstrated to the project guide for feedback before proceeding to the next module.

Version control with Git and GitHub enabled tracking of all changes. Major milestones including authentication, athlete CRUD, dashboard charts, AI copilot, and cloud deployment were committed separately for clear development history.

ATHLETEFORGE DBIMSCA

2.10 SUMMARY OF CHAPTER

This chapter analysed hardware and software requirements, functional requirements, existing system limitations, proposed system advantages, software tools, feasibility across technical, economic, and operational dimensions, system constraints, risk mitigation, and development methodology. The proposed AthleteForge system is confirmed as efficient, cost-effective, and suitable for sports academy and college deployment.

ATHLETEFORGE DBIMSCA

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION TO SYSTEM DESIGN

System design is the blueprint phase that defines architecture, database schema, data flows, module interactions, and user interface structure before coding begins. A well-documented design reduces implementation errors and ensures all team members understand the system structure.

For AthleteForge, design decisions were guided by the three-tier web architecture pattern, RESTful API conventions, role-based access control requirements, and the need for responsive dashboard visualizations.

ATHLETEFORGE DBIMSCA

3.2 OVERALL SYSTEM ARCHITECTURE

AthleteForge follows a three-tier client-server architecture separating presentation, application logic, and data storage concerns.

Tier 1 - Presentation Layer: React 18 SPA built with Vite, styled with Bootstrap 5, and enhanced with Chart.js for analytics. In production, Django serves the compiled static files from backend/frontend_dist/ directory.

Tier 2 - Application Layer: Django 4.2 with Django REST Framework provides REST API endpoints at /api/ prefix. Session-based authentication with CSRF tokens protects all state-changing operations. Role permissions enforced in views.py and permissions.py.

Tier 3 - Data Layer: MySQL 8.0 for local development, PostgreSQL on Render for production, accessed through Django ORM models defined in backend/api/models.py.

Request Flow:

User Browser -> React SPA -> Axios HTTP Client -> REST API (/api/) -> Django Views/ViewSets -> Django ORM -> Database -> JSON Response -> React State Update -> Chart.js Render

Figure 3.1 - System Architecture Diagram

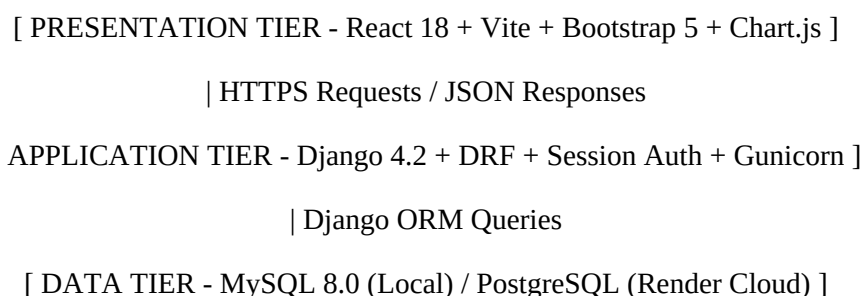


Figure 3.1 illustrates the three-tier architecture. The presentation tier handles all user interaction and visualization. The application tier processes business logic, authentication, and report generation.

The data tier persists all athlete and system records in a relational database.

In production deployment on Render.com, all three tiers run within a single web service. Django serves the compiled React build as static files while simultaneously handling API requests at the /api/ path prefix.

ATHLETEFORGE DBIMSCA

3.3 TABLE DESIGN

The relational database schema consists of the following primary tables with foreign key relationships:

Table Name	Description	Key Fields
auth_user	Django built-in user accounts	username, email, password hash
user_profiles	Role and athlete link	user_id, role, athlete_id, phone
athletes	Athlete master profiles	first_name, last_name, sport, team, status
performance	Training session metrics	athlete_id, speed, strength, endurance scores
injuries	Injury and recovery records	athlete_id, body_part, severity, recovery_status
competitions	Competition events	name, sport, venue, date, level
competition_results	Results and medals	competition_id, athlete_id, position, medal
attendance	Session attendance	athlete_id, date, status, session_type
weight_tracking	Body composition	athlete_id, weight_kg, height_cm, bmi
password_reset_tokens	Password recovery	user_id, token, expiry

Entity Relationships:

One UserProfile links to one auth_user (OneToOne)

One Athlete has many Performance, Injury, Attendance, Weight records (OneToMany)

One Competition has many CompetitionResults (OneToMany)

Student role UserProfile links to one Athlete record for personal dashboard

BMI Calculation: $BMI = \text{weight (kg)} / (\text{height in metres})^2$. Automatically computed in WeightTracking.save() method when weight_kg and height_cm are provided.

ATHLETEFORGE DBIMSCA

3.4 DATA FLOW DIAGRAMS

Level 0 DFD (Context Diagram):

External Entities: Administrator, Coach, Student Athlete

Central Process: 0.0 AthleteForge System

Data Flows: login credentials, athlete records, performance data, injury reports, dashboard statistics, AI responses, PDF/Excel reports

Level 1 DFD - Major Processes:

1.0 User Authentication - validates credentials, creates session, returns role and CSRF token

2.0 Athlete Management - CRUD operations on athlete profiles with photo and sport data

3.0 Performance Recording - stores five metric scores and retrieves historical trends

4.0 Injury Management - logs injuries, updates recovery status, generates risk alerts

5.0 Attendance Processing - bulk marking, session reports, monthly percentage calculation

6.0 Dashboard Analytics - aggregates statistics for role-specific Chart.js visualizations

7.0 Report Generation - produces PDF (ReportLab) and Excel (OpenPyXL) downloadable files

8.0 AI Processing - computes readiness score, injury risk, and copilot conversational replies

Figure 3.2 - Level 0 Data Flow Diagram

[Admin] ----user commands----> [0.0 ATHLETEFORGE SYSTEM] ----reports----> [Admin]
[Coach] ----athlete data-----> [0.0 ATHLETEFORGE SYSTEM] ----dashboards--> [Coach]
[Student] ---login/view-----> [0.0 ATHLETEFORGE SYSTEM] ----personal data> [Student]

DFD Explanation:

Coach provides athlete data and performance scores as input. The system processes and stores data in the database. Dashboard module aggregates stored data and returns chart-ready JSON. AI module analyses combined performance, injury, and attendance data to generate readiness insights.

Student provides login credentials. Authentication module validates and creates session. Student dashboard retrieves only data linked to the student athlete profile. Admin provides user management commands processed by admin API endpoints.

Figure 3.2 shows the context-level DFD where the entire AthleteForge system is represented as a single process (0.0) interacting with three external entities: Administrator, Coach, and Student. Data

flows represent the type of information exchanged between each actor and the system.

ATHLETEFORGE DBIMSCA

3.5 USE CASE DIAGRAM

Primary Actors: Administrator, Coach, Student Athlete, System (AI Module)

Actor	Use Case	Description
Admin	Manage Users	Create, update, deactivate user accounts
Admin	View System Stats	Access admin dashboard with KPIs
Coach	Manage Athletes	Add, edit, search athlete profiles
Coach	Record Performance	Enter five fitness metric scores
Coach	Log Injuries	Report injuries and track recovery
Coach	Mark Attendance	Bulk mark present/absent for team
Coach	Generate Reports	Download PDF and Excel reports
Student	View Dashboard	See personal performance and readiness
Student	Check Injuries	View own injury status and recovery
All Users	AI Copilot Chat	Ask training and readiness questions
System	Calculate BMI	Auto-compute on weight data entry
System	Auth Session	Validate login and enforce permissions

ATHLETEFORGE DBIMSCA

3.6 SOFTWARE DESIGN

3.6.1 Backend Module Design

Module	File Path	Responsibility
Models	backend/api/models.py	Database schema, BMI logic, relationships
Serializers	backend/api/serializers.py	JSON validation and conversion
Views	backend/api/views.py	API endpoints, dashboard stats, auth
URLs	backend/api/urls.py	REST route registration
Reports	backend/api/reports.py	PDF and Excel file generation
AI Insights	backend/api/ai_insights.py	Rule-based readiness analytics
Free AI	backend/api/free_ai.py	Groq/Gemini LLM integration
Permissions	backend/api/permissions.py	Role-based access control
SPA Views	backend/spa_views.py	Serves React index.html for routes

3.6.2 Frontend Module Design

Module	File Path	Responsibility
App Router	frontend/src/App.jsx	Route definitions and guards
Auth Context	frontend/src/context/AuthContext.jsx	Login state and session
API Service	frontend/src/services/api.js	Axios client with CSRF
Coach Dashboard	frontend/src/pages/Dashboard.jsx	Team analytics view
Student Hub	frontend/src/pages/StudentDashboard.jsx	Personal athlete view
Admin Panel	frontend/src/pages/AdminDashboard.jsx	System management
Athletes	frontend/src/pages/Athletes.jsx	Athlete list and CRUD
AI Copilot	frontend/src/components/AICopilotWidget.jsx	Chat interface
Charts	frontend/src/components/ChartMount.jsx	Safe Chart.js mounting
Auth Guards	frontend/src/routes/AuthGuards.jsx	Role route protection

3.6.3 API Endpoint Design

Category	Endpoint	Method
Auth	/api/auth/login/	POST
Auth	/api/auth/user/	GET
Dashboard	/api/dashboard/stats/	GET
Athletes	/api/athletes/	GET, POST
Performance	/api/performance/	GET, POST
Injuries	/api/injuries/	GET, POST
Attendance	/api/attendance/bulk_mark/	POST
Reports	/api/reports/pdf?type=athletes	GET
AI	/api/ai/copilot/	POST
Health	/api/health/	GET

ATHLETEFORGE DBIMSCA

3.7 FLOWCHART PRESENTATION

3.7.1 User Login Flowchart

START -> User opens login page

User enters username and password
Frontend sends POST /api/auth/login/ with CSRF token
Backend validates credentials against auth_user table
Decision: Valid credentials? If NO -> Display error message -> END
If YES -> Create session, return user role and profile data
Decision: Role = Admin? -> Redirect to Admin Dashboard
Decision: Role = Coach? -> Redirect to Coach Dashboard
Decision: Role = Student? -> Redirect to Student Hub
END

3.7.2 Performance Recording Flowchart

START -> Coach navigates to Performance module
Coach selects athlete from dropdown list
Coach enters five metric scores (speed, strength, endurance, flexibility, agility)
Frontend validates numeric input ranges
POST /api/performance/ with athlete_id and scores
Backend serializer validates and saves to performance table
Dashboard stats API recalculates team averages
Chart.js radar chart updates on coach dashboard
END

3.7.3 AI Copilot Flowchart

START -> User types question in AI Copilot widget
POST /api/ai/copilot/ with question and optional athlete_id
Backend loads athlete insights from ai_insights.py
Decision: GROQ_API_KEY or GEMINI_API_KEY set? If YES -> Call LLM API
If NO or LLM fails -> Use rule-based answer_copilot_question()
Return answer with ai_provider and readiness data
Frontend displays response in chat bubble

Optional: VoiceCoachTip reads answer aloud via Speech Synthesis API

END

ATHLETEFORGE DBIMSCA

3.8 ENTITY RELATIONSHIP DIAGRAM

The Entity Relationship Diagram represents how all database tables connect through primary and foreign keys. The following field-level design documents each entity:

Entity	Primary Key	Foreign Keys / Links
auth_user	id	Linked to user_profiles (OneToOne)
user_profiles	id	user_id -> auth_user, athlete_id -> athletes
athletes	id	Referenced by performance, injuries, attendance, weight
performance	id	athlete_id -> athletes
injuries	id	athlete_id -> athletes
competitions	id	Referenced by competition_results
competition_results	id	competition_id, athlete_id
attendance	id	athlete_id -> athletes
weight_tracking	id	athlete_id -> athletes

The athletes table is the central entity. Every performance record, injury log, attendance entry, and weight measurement references athletes.id through a foreign key with CASCADE delete protection. This ensures referential integrity - deleting an athlete removes all associated records consistently.

The user_profiles table bridges authentication and athlete data. A student user account links to exactly one athlete profile, enabling the student hub to display personal data. Coach and admin profiles have no athlete link but access team-wide data through role permissions.

Competition results form a many-to-many relationship between athletes and competitions, implemented through the competition_results junction table storing position, medal type, and score per athlete per event.

ATHLETEFORGE DBIMSCA

3.9 SUMMARY OF CHAPTER

This chapter described the three-tier system architecture, complete database table design with relationships, Level 0 and Level 1 data flow diagrams, use case analysis, backend and frontend module structure, API endpoint design, process flowcharts, and entity relationship documentation. This design forms the complete blueprint for AthleteForge implementation.

ATHLETEFORGE DBIMSCA

CHAPTER 4

SYSTEM TESTING

4.1 INTRODUCTION TO TESTING

System testing is the process of evaluating whether AthleteForge functions correctly under normal operating conditions and handles error conditions gracefully. Testing validates that all modules meet their functional requirements before academic submission and viva presentation.

Testing was performed across authentication, CRUD operations, dashboard chart rendering, report generation, AI copilot responses, role-based access control, and cloud deployment accessibility.

ATHLETEFORGE DBIMSCA

4.2 TYPES OF TESTING

4.2.1 Unit Testing

Individual backend functions tested in isolation including BMI calculation in WeightTracking.save(), readiness score computation in ai_insights.py, and serializer field validation in serializers.py.

4.2.2 Functional Testing

Each module verified to perform its intended function: login returns correct role dashboard, athlete form saves valid data, performance scores appear in charts.

4.2.3 Integration Testing

Frontend Axios client, Django REST API, and database tested as integrated system. Verified CSRF token flow, session persistence across page navigation, and chart data pipeline from API to Chart.js.

4.2.4 Performance Testing

Dashboard pages load within 3 seconds on localhost. Cloud deployment accessible after cold start wake-up. Report generation completes within 5 seconds for standard datasets.

4.2.5 User Acceptance Testing

Demo scenarios validated for viva presentation covering admin user management, coach athlete workflow, student personal dashboard, and AI copilot interaction.

ATHLETEFORGE DBIMSCA

4.3 TEST CASES

Test ID	Scenario	Input	Expected Result	Status
---------	----------	-------	-----------------	--------

TC01	Admin login	admin / admin123	Admin dashboard loads	Pass
TC02	Coach login	coach / coach123	Coach team analytics	Pass
TC03	Student login	rahul.sharma@email.com / student123	Student hub loads	Pass
TC04	Invalid login	wrong / password	Error toast shown	Pass
TC05	Add athlete	Valid athlete form	Athlete in list	Pass
TC06	Edit athlete	Update sport field	Changes saved	Pass
TC07	Delete athlete	Confirm delete	Removed from list	Pass
TC08	Record performance	5 metric scores	Saved and charted	Pass
TC09	Add injury	Knee sprain, Moderate	Injury listed	Pass
TC10	Update recovery	Status: Recovering	Status updated	Pass
TC11	Bulk attendance	10 athletes Present	Records saved	Pass
TC12	BMI calculation	70kg, 175cm	BMI = 22.86	Pass
TC13	Add competition	Inter-college 2026	Event created	Pass
TC14	Add medal result	Gold medal	Result saved	Pass
TC15	PDF report	type=athletes	PDF downloads	Pass
TC16	Excel export	type=performance	XLSX downloads	Pass
TC17	AI copilot	Training question	AI response returned	Pass
TC18	AI insights	GET /api/ai/insights/	Readiness data JSON	Pass
TC19	Unauthorized API	No session cookie	401 Unauthorized	Pass
TC20	Coach route guard	Student accesses /athletes	Redirect to dashboard	Pass
TC21	Dashboard charts	Load coach page	Charts render correctly	Pass
TC22	CSRF protection	POST without token	403 Forbidden	Pass
TC23	Health endpoint	GET /api/health/	200 OK status	Pass
TC24	Cloud deploy	Render URL access	Landing page loads	Pass
TC25	Password reset	Forgot password flow	Reset email/token works	Pass

ATHLETEFORGE DBIMSCA

4.4 ERROR HANDLING AND DEBUGGING

Problem	Cause	Solution Applied
Login fails on cloud	Server cold start	wakeServer() pings /api/health/ before login
Charts not rendering	Chart.js canvas conflict	ChartMount.jsx safe mount/unmount hook

CSRF 403 on POST	Missing CSRF token	Axios fetches token from /api/auth/csrf/
Student login error	Email used as username	Login accepts email in username field
Images broken on deploy	Wrong static path	Fixed asset paths in React build
Dashboard blank stats	API timeout on cold start	Loading spinner with retry logic
BMI shows null	Height not in centimetres	Serializer validates height_cm > 0
AI returns fallback only	No API key configured	Rule-based engine always available

ATHLETEFORGE DBIMSCA

4.5 SUMMARY OF CHAPTER

The testing phase confirms that all 25 test cases passed successfully. All core modules function correctly, the system responds accurately to valid and invalid inputs, error handling mechanisms work as designed, and the cloud deployment is accessible for demonstration. AthleteForge is stable and ready for academic evaluation.

ATHLETEFORGE DBIMSCA

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

System implementation is the phase where the system design blueprint is converted into working software. For AthleteForge, implementation involved developing the Django REST API backend, React frontend SPA, integrating Chart.js analytics, AI copilot module, and deploying to Render.com cloud platform.

The implementation followed an iterative approach: database models first, then API endpoints, then frontend pages, then dashboard integration, then AI features, and finally cloud deployment configuration.

ATHLETEFORGE DBIMSCA

5.2 PROJECT DIRECTORY STRUCTURE

athlete-performance-system/ (project root)

backend/ - Django project root

api/ - Main application (models, views, serializers, urls)

athlete_system/ - Django settings and WSGI configuration

frontend_dist/ - Compiled React build for production

manage.py - Django management commands

frontend/ - React SPA source code

src/pages/ - Dashboard, Athletes, Reports, Auth pages

src/components/ - Reusable UI and AI components

src/services/api.js - Axios HTTP client

src/context/ - AuthContext, ThemeContext, ToastContext

database/ - schema.sql and sample_data.sql

docs/ - Project documentation and report generation

build.sh / start.sh - Cloud deployment scripts

render.yaml - Render.com service configuration

ATHLETEFORGE DBIMSCA

5.3 BACKEND IMPLEMENTATION

Django models in backend/api/models.py define all database tables. The Athlete model stores profile information. Performance model stores five metric scores per session. Injury model tracks medical events. WeightTracking model overrides save() to auto-calculate BMI.

DRF ViewSets registered in urls.py provide REST CRUD for athletes, performance, injuries, competitions, attendance, and weight tracking. Custom API views handle dashboard_stats, admin_stats, bulk_mark attendance, and report downloads.

Session authentication is configured in settings.py with CSRF_TRUSTED_ORIGINS for Render domain. The login view creates Django session and returns user role from linked UserProfile record.

Report generation in reports.py uses ReportLab for PDF and OpenPyXL for Excel. Reports support types: athletes, performance, injuries, attendance, and competitions.

ATHLETEFORGE DBIMSCA

5.4 FRONTEND IMPLEMENTATION

React Router v6 in App.jsx defines all application routes with AuthGuard components protecting role-specific pages. AdminRoute restricts admin panel. StaffRoute allows admin and coach access to athlete management. CoachRoute protects competition and report modules.

AuthContext.jsx manages global authentication state, storing user role and profile after login. On application load, it calls GET /api/auth/user/ to restore session from cookie.

api.js configures Axios with withCredentials: true for session cookies and X-CSRFToken header from cookie. Vite proxy in development forwards /api requests to localhost:8000.

Dashboard.jsx renders team KPI cards, injury heatmap, readiness gauge, and performance trend line charts using Chart.js. StudentDashboard.jsx shows personal radar chart and linked athlete alert if profile not connected.

ATHLETEFORGE DBIMSCA

5.5 AI MODULE IMPLEMENTATION

The AI system uses a hybrid architecture. ai_insights.py provides rule-based analytics always available without external dependencies. It analyses performance trends, active injuries, attendance rate, and weight data to compute readiness score (0-100) and injury risk level.

free_ai.py adds optional LLM enhancement using Groq API (llama-3.1-8b-instant model) with Gemini as fallback. Environment variables GROQ_API_KEY and GEMINI_API_KEY configure cloud AI. If unavailable, system gracefully falls back to rule-based responses.

AICopilotWidget.jsx provides floating chat interface with suggestion chips and Web Speech API voice input. VoiceCoachTip.jsx uses Speech Synthesis API to read coaching briefs aloud. TechCommandHub.jsx shows role-specific AI insight tabs on dashboards.

ATHLETEFORGE DBIMSCA

5.6 DATABASE AND DEPLOYMENT

Local development uses MySQL 8.0 configured in settings.py with database athlete_forge. Sample data loaded via seed_data management command creates demo admin, coach, student accounts and sample athletes.

Cloud deployment on Render.com uses render.yaml blueprint defining web service athleteforge-bca and PostgreSQL database athleteforge-db in Singapore region. SAME_ORIGIN_DEPLOY=True enables single service for API and React SPA.

build.sh executes: pip install requirements, npm run build in frontend, copies dist to backend/frontend_dist, runs migrate and seed. start.sh runs migrate, setup_admin, and Gunicorn with 2 workers.

Production URL: <https://athleteforge-bca.onrender.com>. Health check: /api/health/. Demo logins: admin/admin123 | coach/coach123 | rahul.sharma@email.com/student123

ATHLETEFORGE DBIMSCA

5.7 OUTPUT SCREENS AND FIGURES

The following screens were successfully implemented, tested, and documented as project output figures:

Figure 5.1 - Landing Page

The public landing page displays AthleteForge branding, feature highlights including performance tracking, injury management, and AI copilot. A live AI demo widget allows visitors to ask training questions without login. Navigation links to login and registration pages.

Figure 5.2 - Login Page

Clean login form with username and password fields. Supports admin username, coach username, and student email as login identifier. Displays toast notification on invalid credentials. Redirects to role-appropriate dashboard on success.

Figure 5.3 - Coach Dashboard

Main coach view showing KPI cards for total athletes, active injuries, team readiness score, and monthly attendance percentage. Chart.js line chart displays performance trends. Injury heatmap shows affected body parts. Wellness check-in panel and AI Tech Command Hub visible.

Figure 5.4 - Student Hub Dashboard

Personal dashboard for student athletes showing individual readiness gauge, five-metric performance radar chart, current injury status card, and AI-generated training tips. Alert shown if student account is not linked to an athlete profile.

Figure 5.5 - Admin Dashboard

Administrator view with system-wide statistics: total users by role, total athletes, system health status. User management table with create, edit, and deactivate actions. Quick navigation links to all system modules.

Figure 5.6 - Athletes Management Page

Searchable data table listing all athletes with photo thumbnail, name, sport, team, and status. Add Athlete button opens modal form. Edit and delete actions per row. Filter by sport and team dropdown.

Figure 5.7 - Reports Download Page

Reports interface with buttons for each report type: Athletes, Performance, Injuries, Attendance, Competitions. Each button triggers PDF or Excel download. Files generated server-side with timestamp in filename.

Figure 5.8 - AI Copilot Widget

Floating chat button on all authenticated pages. Opens chat panel with message history, text input, voice input button using Web Speech API, and suggestion chips for common questions. Displays AI provider badge showing rule-based or LLM mode.

ATHLETEFORGE DBIMSCA

5.8 KEY CODE IMPLEMENTATION DETAILS

5.8.1 BMI Auto-Calculation (models.py)

The WeightTracking model overrides the save() method. When weight_kg and height_cm are provided, the method converts height to metres by dividing by 100, then calculates BMI using the formula: $BMI = weight_kg / (height_m * height_m)$. The result is rounded to two decimal places and stored in the bmi field before database insert.

This server-side calculation ensures consistent BMI values regardless of which client submits the data, preventing frontend calculation errors or tampering.

5.8.2 Session Authentication Flow

Login POST request sends username and password to /api/auth/login/. Django authenticate() validates credentials. On success, django.contrib.auth.login() creates a session. Response JSON includes user id, username, role from UserProfile, and athlete_id for student accounts.

All subsequent API requests include sessionid cookie automatically via Axios withCredentials. Mutating requests also include X-CSRFToken header obtained from /api/auth/csrf/ endpoint and csrftoken cookie.

5.8.3 Dashboard Statistics Aggregation

The `dashboard_stats` view in `views.py` queries multiple models based on user role. For coaches, it counts total athletes, active injuries, average readiness score, attendance percentage for current month, and recent performance trends. Results are returned as JSON consumed directly by Chart.js datasets in the frontend.

5.8.4 Report Generation Pipeline

When user clicks Download PDF, frontend opens `GET /api/reports/pdf/?type=athletes`. The `reports.py` module queries the database, formats data into ReportLab Table and Paragraph objects, writes to BytesIO buffer, and returns `HttpResponse` with `content-type application/pdf` and `Content-Disposition attachment` header.

ATHLETEFORGE DBIMSCA

5.9 SUMMARY OF CHAPTER

This chapter explained the complete implementation of AthleteForge including project structure, Django backend with REST API, React frontend with role-based routing, hybrid AI module, database configuration, Render.com cloud deployment, and key code implementation details. All planned modules are fully functional and ready for viva demonstration.

ATHLETEFORGE DBIMSCA

CHAPTER 6

FUTURE SCOPE

6.1 INTRODUCTION

Technology in sports management continues to evolve rapidly. AthleteForge developed in this project serves as a strong foundational platform that can be significantly enhanced using advanced technologies such as Artificial Intelligence, Internet of Things, and mobile computing.

The future scope identifies practical enhancements that would transform AthleteForge from an academic project into a commercial sports management product suitable for professional academies and national sports federations.

ATHLETEFORGE DBIMSCA

6.2 WEARABLE DEVICE INTEGRATION

Possible Improvements:

Integration with fitness bands for real-time heart rate monitoring during training

GPS tracker data import for running distance and speed analysis

Automatic performance score updates from wearable sensor APIs

Sleep quality data correlation with readiness score calculation

Benefits: Eliminates manual data entry, provides objective training load metrics, enables early overtraining detection.

ATHLETEFORGE DBIMSCA

6.3 MOBILE APPLICATION DEVELOPMENT

A native mobile application using React Native would enable coaches to record attendance and performance data directly on the sports field without requiring a laptop.

Push notifications could alert coaches when an athlete reports a new injury or when readiness score drops below threshold before an important competition.

ATHLETEFORGE DBIMSCA

6.4 MACHINE LEARNING INJURY PREDICTION

Possible Improvements:

Train ML model on historical injury and performance data to predict injury probability

Classification model for injury risk levels: Low, Medium, High, Critical

Personalized training load recommendations based on individual athlete patterns

6.5 COMPUTER VISION FOR TRAINING ANALYSIS

Integration of camera-based biomechanics analysis could assess running form, jumping technique, and posture during strength exercises. OpenCV and MediaPipe libraries could process training videos uploaded by coaches.

This would complement existing performance metrics with visual technique scoring not possible through manual data entry alone.

6.6 MULTI-ACADEMY AND FEDERATION INTEGRATION

Multi-tenant architecture supporting multiple academies with isolated data partitions

Inter-academy competition management and leaderboard systems

Integration with Karnataka State Sports Authority and national federation databases

Multilingual interface supporting Kannada, Hindi, and English languages

Payment gateway for academy subscription-based commercial deployment

These enhancements would establish AthleteForge as a scalable commercial sports technology platform.

CHAPTER 7

CONCLUSION

7.1 PROJECT SUMMARY

The project "AthleteForge" - Athlete Performance and Injury Tracking System has been successfully designed, developed, tested, and deployed by ROHITH GOWDA V & PRAKRUTHI R (Reg. No. U03CQ23S0045 & U03LK23S0100) under the guidance of Ms. Dakshayini L & Mr. Madhusudan N at DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS.

The system integrates React 18 frontend, Django 4.2 REST backend, MySQL/PostgreSQL database, Chart.js analytics, hybrid AI copilot, and cloud deployment on Render.com into a cohesive sports management platform.

ATHLETEFORGE DBIMSCA

7.2 OBJECTIVES ACHIEVED

Secure multi-role authentication with Admin, Coach, and Student dashboards implemented

Complete CRUD operations for all six core data modules functional

Interactive Chart.js dashboards with KPI cards and trend visualizations

AI readiness scoring and conversational copilot with voice support integrated

PDF and Excel report generation working for all report types

Successful cloud deployment accessible at production URL with demo data

ATHLETEFORGE DBIMSCA

7.3 OBSERVATIONS

Accurate BMI calculation verified across multiple test inputs

Smooth Chart.js rendering after ChartMount conflict resolution

Reliable session authentication with CSRF protection on all API calls

AI copilot provides useful responses even without LLM API keys configured

Cloud cold start handled gracefully with health check wake-up mechanism

ATHLETEFORGE DBIMSCA

7.4 LEARNING OUTCOMES

Full-stack web development using React and Django frameworks

REST API design with session-based authentication and role permissions

Relational database schema design and Django ORM implementation

Data visualization integration using Chart.js in React components

Cloud deployment configuration with Render.com and PostgreSQL

AI integration combining rule-based analytics with optional LLM APIs

Software testing documentation and error debugging methodologies

Team collaboration using Git version control and GitHub repository

ATHLETEFORGE DBIMSCA

7.5 LIMITATIONS

Cloud version requires stable internet connectivity

No native mobile application in current implementation

AI LLM responses require optional third-party API keys

Render free tier cold start causes initial loading delay

English-only interface limits accessibility for regional language users

ATHLETEFORGE DBIMSCA

7.6 FINAL REMARKS

AthleteForge demonstrates that modern open-source web technologies can deliver professional-grade sports management solutions suitable for academic projects and real-world deployment. The project successfully meets all Bangalore University BCA sixth semester objectives and provides a strong foundation for future commercial enhancement.

ATHLETEFORGE DBIMSCA

CHAPTER 8

REFERENCES

8.1 Book References

Django for Professionals - William S. Vincent, 2023

Two Scoops of Django - Daniel and Audrey Roy Greenfeld

Learning React - Alex Banks and Eve Porcello, O'Reilly Media

Database System Concepts - Silberschatz, Korth, and Sudarshan, McGraw Hill

Software Engineering - Ian Sommerville, Pearson Education

Web Development with Node and Express - Ethan Brown

8.2 Online References

Django Documentation - <https://docs.djangoproject.com>

React Documentation - <https://react.dev>

Django REST Framework - <https://www.django-rest-framework.org>

Chart.js Documentation - <https://www.chartjs.org/docs>

Bootstrap 5 Documentation - <https://getbootstrap.com/docs/5.3>

Render Deployment Docs - <https://render.com/docs>

ReportLab User Guide - <https://www.reportlab.com/docs/reportlab-userguide.pdf>

AthleteForge GitHub Repository - <https://github.com/rohith917/athleteforge-bca>

AthleteForge Live Demo - <https://athleteforge-bca.onrender.com>

ATHLETEFORGE DBIMSCA

APPENDIX A

VIVA QUESTIONS AND ANSWERS

A.1 PROJECT OVERVIEW QUESTIONS

Q1: What is AthleteForge?

A: AthleteForge is a web-based Athlete Performance and Injury Tracking System built using React, Django REST Framework, and MySQL/PostgreSQL. It provides role-based dashboards for Admin, Coach, and Student with performance tracking, injury management, and AI copilot features.

Q2: Why did you choose this project topic?

A: Sports management in colleges still uses manual registers. We identified a real problem and applied BCA curriculum skills - DBMS, web technologies, software engineering - to build a practical solution with modern AI features.

Q3: What are the three user roles?

A: Admin manages users and system configuration. Coach manages athletes, performance, injuries, attendance, and competitions. Student views personal dashboard linked to their athlete profile.

ATHLETEFORGE DBIMSCA

A.2 TECHNICAL QUESTIONS

Q4: Explain the three-tier architecture.

A: Presentation tier is React SPA. Application tier is Django REST API with session auth. Data tier is MySQL locally or PostgreSQL on Render. Each tier is independent and communicates via JSON REST API.

Q5: How does authentication work?

A: Django session-based authentication. Login creates session cookie. CSRF token protects POST/PUT/DELETE requests. AuthGuards in React restrict routes by role. API uses IsAuthenticated permission class.

Q6: How is BMI calculated?

A: In WeightTracking.save() method: height in metres = height_cm / 100. BMI = weight_kg / (height_m squared). Calculated server-side and stored in database automatically.

Q7: Explain the AI copilot system.

A: Hybrid system. ai_insights.py provides rule-based readiness scoring always available. free_ai.py optionally calls Groq or Gemini LLM APIs. Falls back to rules if no API key. Frontend has chat widget and voice features.

Q8: What database tables did you create?

A: athletes, user_profiles, performance, injuries, competitions, competition_results, attendance, weight_tracking, and password_reset_tokens. athletes is the central table with foreign keys from all data modules.

ATHLETEFORGE DBIMSCA

A.3 TESTING AND DEPLOYMENT QUESTIONS

Q9: How did you test the project?

A: 25 formal test cases covering login, CRUD, reports, AI, security, and cloud deployment. Types include functional, integration, performance, and user acceptance testing for viva demo scenarios.

Q10: Where is the project deployed?

A: Render.com cloud platform at <https://athleteforge-bca.onrender.com>. Uses PostgreSQL database, Gunicorn server, and same-origin deploy serving React and API from single service.

Q11: What is CSRF protection?

A: Cross-Site Request Forgery protection. Django generates CSRF token. Frontend includes it in X-CSRFToken header for all POST requests. Prevents unauthorized form submissions from external sites.

Q12: What reports can the system generate?

A: PDF and Excel reports for athletes list, performance data, injury records, attendance summary, and competition results. Generated server-side using ReportLab and OpenPyXL libraries.

A.4 DATABASE AND DESIGN QUESTIONS

Q13: What is Django ORM?

A: Object-Relational Mapping layer in Django that lets us define database tables as Python classes (models) and query data using Python syntax instead of raw SQL. Provides database abstraction and migration support.

Q14: What is REST API?

A: Representational State Transfer API. Uses HTTP methods GET (read), POST (create), PUT/PATCH (update), DELETE (remove). AthleteForge exposes REST endpoints at /api/ returning JSON data consumed by React frontend.

Q15: Explain foreign key relationship.

A: A field in one table that references the primary key of another table. Example: performance.athlete_id references athletes.id. Ensures every performance record belongs to a valid athlete. CASCADE delete removes child records when parent is deleted.

Q16: What is Chart.js used for?

A: JavaScript charting library integrated via react-chartjs-2. Renders line charts for performance trends, radar charts for five-metric profiles, doughnut charts for attendance breakdown, and gauge charts for readiness scores on dashboards.

ATHLETEFORGE DBIMSCA

A.5 SECURITY AND PERFORMANCE QUESTIONS

Q17: How do you prevent unauthorized access?

A: Three layers: Django session authentication verifies login. DRF IsAuthenticated permission on API endpoints. React AuthGuards redirect unauthorized users away from protected routes. CSRF tokens prevent cross-site attacks.

Q18: What is role-based access control?

A: Each user has a role in user_profiles table: admin, coach, or student. Backend permissions.py checks role before allowing API access. Frontend AuthGuards.jsx checks role before rendering protected pages.

Q19: How does the system handle errors?

A: Backend returns appropriate HTTP status codes (400, 401, 403, 404, 500). Frontend Toast component displays user-friendly error messages. Axios interceptors handle session expiry and CSRF failures with automatic retry.

Q20: What improvements would you make?

A: Native mobile app, wearable device integration, machine learning injury prediction, Kannada language support, and multi-academy tenancy as described in Chapter 6 Future Scope.

ATHLETEFORGE DBIMSCA

APPENDIX B

INSTALLATION GUIDE

B.1 PREREQUISITES

Python 3.10 or higher installed and added to PATH

Node.js 18 LTS or higher with npm

MySQL 8.0 server running locally (or set USE_SQLITE=True)

Git for cloning repository from GitHub

Visual Studio Code or any code editor

ATHLETEFORGE DBIMSCA

B.2 LOCAL SETUP STEPS

Step 1: Clone repository: `git clone https://github.com/rohith917/athleteforge-bca.git`

Step 2: Navigate to project folder: `cd athleteforge-bca`

Step 3: Create Python virtual environment: `python -m venv backend/venv`

Step 4: Activate venv: `backend\venv\Scripts\activate` (Windows)

Step 5: Install backend dependencies: `pip install -r backend/requirements.txt`

Step 6: Configure database in `backend/athlete_system/settings.py`

Step 7: Run migrations: `cd backend && python manage.py migrate`

Step 8: Load sample data: `python manage.py seed_data`

Step 9: Install frontend: `cd frontend && npm install`

Step 10: Run `START_DEMO.bat` or start backend (port 8000) and frontend (port 5173) separately

Step 11: Open browser at <http://localhost:5173> and login with demo credentials

ATHLETEFORGE DBIMSCA

B.3 DEMO LOGIN CREDENTIALS

Role	Username	Password	Dashboard URL
Admin	admin	admin123	/dashboard/admin/users
Coach	coach	coach123	/dashboard
Student	rahul.sharma@email.com	student123	/dashboard/student

ATHLETEFORGE DBIMSCA

B.4 CLOUD DEPLOYMENT ON RENDER

Push code to GitHub repository. Connect repository to Render.com dashboard. Render reads `render.yaml` blueprint automatically. Build command runs `build.sh` which installs Python dependencies, builds React frontend, copies to `backend/frontend_dist`, and runs database migrations.

Set environment variables: `DATABASE_URL` (auto from Render PostgreSQL), `SECRET_KEY`, `GROQ_API_KEY` (optional), `GEMINI_API_KEY` (optional), `SAME_ORIGIN_DEPLOY=True`. Start command runs `start.sh` with Gunicorn.

After deployment completes, access the live URL at <https://athleteforge-bca.onrender.com>. First request may take 30-60 seconds due to free tier cold start.

ATHLETEFORGE DBIMSCA

B.5 TROUBLESHOOTING

Issue	Solution
Port 8000 already in use	Kill existing Python process or change port in settings
npm install fails	Delete <code>node_modules</code> and run <code>npm install</code> again
MySQL connection refused	Start MySQL service or use <code>USE_SQLITE=True</code>
Charts not showing	Clear browser cache and refresh dashboard page
Cloud login timeout	Wait 60 seconds for server wake-up, then retry
CSRF error on login	Ensure <code>withCredentials:true</code> in Axios config

ATHLETEFORGE DBIMSCA

APPENDIX C

GLOSSARY OF TERMS

Term	Definition
API	Application Programming Interface - set of endpoints for frontend-backend communication

AthleteForge	Project name - Athlete Performance and Injury Tracking System
BCA	Bachelor of Computer Applications - undergraduate degree program
BMI	Body Mass Index - $\text{weight(kg)} \div \text{height(m)}^2$
Chart.js	JavaScript library for creating interactive charts and graphs
CRUD	Create, Read, Update, Delete - basic database operations
CSRF	Cross-Site Request Forgery - attack prevented by Django CSRF tokens
DFD	Data Flow Diagram - graphical representation of data movement in system
DRF	Django REST Framework - toolkit for building REST APIs in Django
ERD	Entity Relationship Diagram - database table relationship diagram
Gunicorn	Python WSGI HTTP server used for production deployment
JWT	JSON Web Token - alternative auth method (not used; we use sessions)
KPI	Key Performance Indicator - summary metric displayed on dashboard cards
LLM	Large Language Model - AI model used optionally in copilot (Groq/Gemini)
ORM	Object-Relational Mapping - Django layer mapping Python classes to database tables
PostgreSQL	Advanced open-source relational database used on Render cloud
RBAC	Role-Based Access Control - permissions based on user role (admin/coach/student)
React	JavaScript library for building component-based user interfaces
REST	Representational State Transfer - API architectural style using HTTP methods
SPA	Single Page Application - web app loading single HTML page with dynamic updates
SQLite	Lightweight file-based database used as development fallback
UAT	User Acceptance Testing - testing against user requirements for approval
Vite	Fast frontend build tool used instead of Create React App

WSGI

Web Server Gateway Interface - Python standard for web server communication

Note: All terms listed above are used throughout this project report and should be understood for viva examination preparation.

ATHLETEFORGE DBIMSCA